

Cliente Identificación del problema y análisis de requerimientos

Caso de Estudio:

Cliente Usuario	Universidad Icesi – Facultad de Negocios y Economía Estudiantes de Economía y Negocios Internacionales
Requerimientos funcionales	<i>RF1: Colocación de barcos</i> <i>RF2: Ataque humano</i> <i>RF3: Ataque máquina</i> <i>RF4: Mostrar tableros</i> <i>RF5: Determinar ganador</i> <i>RF6: Partidas múltiples</i> <i>RF7: Modos de juego (estándar/personalizada)</i>
Contexto del problema	<i>La Universidad Icesi, por medio de su Facultad de Negocios y Economía, busca desarrollar un simulador avanzado en 2D del juego BattleShip como apoyo pedagógico para los estudiantes de Economía y Negocios Internacionales. Este proyecto tiene como propósito principal facilitar el análisis de situaciones estratégicas en juegos, modelar dinámicas de competencia entre distintos agentes y explorar patrones en la toma de decisiones.</i> <i>El simulador se plantea como una herramienta complementaria para fortalecer el aprendizaje en teoría de juegos y estrategias de decisión, permitiendo representar y experimentar diversos escenarios competitivos.</i>
Requerimientos no funcionales	RNF1: Fácil de usar, claro y organizado RNF2: Disponibilidad 24/7 RNF3: Compatible con otros dispositivos
Requerimientos de proceso	Uso de GitHub Implementación en Java con POO Documentación con JavaDoc

Identificador y nombre	<i>RF1: Colocación de barcos</i>		
Resumen	<i>Este requerimiento permite al jugador humano colocar sus barcos en el tablero según las reglas del juego. El sistema debe validar que las posiciones no se solapen y estén dentro de los límites del tablero.</i>		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	fila	int	<i>(1-10)</i> <i>Ej. 1</i>

	columna	int	(1-10) Ej: 2
	horizontal	boolean	“¿Horizontal? (true/false)” Ej: true
	tamaño	int	(1-5) Ej 2
Resultado o Postcondición	El barco queda colocado en el tablero humano en la posición especificada si es válida.		
Salidas	Nombre salida	Tipo de dato	Formato
	Mensaje de éxito	String	“Tu tablero: ” 0011000000 0000000000 0000000000 0000000000 0000000000 0000000000 0000000000 0000000000 0000000000 0000000000
	Mensaje de error	String	“NO se pudo agregar ese barco, intenta de nuevo”

Identificador y nombre	RF2: Ataque humano
Resumen	Este requerimiento permite al jugador humano atacar una coordenada en el tablero de la máquina, y también debe validar que la coordenada no haya sido atacada previamente.

Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	fila	int	(1-10) Ej. 2
	columna	int	(1-10) Ej. 3
Resultado o Postcondición	El tablero de la máquina se actualiza con el resultado del ataque		
Salidas	Nombre salida	Tipo de dato	Formato
	Mensaje de éxito	String	Ej. "Atacarás el punto exacto de la fila X, columna Y"
	Mensaje de error1	String	Ej. "Coordenadas inválidas, intenta de nuevo"
	Mensaje de error2	String	El: "Ya disparaste ahí, elegí otra coordenada"

Identificador y nombre	RF3: Ataque máquina		
Resumen	Este requerimiento permite a la máquina realizar un ataque aleatorio en el tablero del jugador humano, evitando coordenadas que ya se han atacado.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
Resultado o Postcondición	El tablero humano es actualizado con el resultado del ataque.		
Salidas	Nombre salida	Tipo de dato	Formato
	Mensaje de éxito	String	"BARCO HUNDIDOOO" "El tiro ha sido acertado" "El tiro ha sido fallado"

Identificador y nombre	RF4: Mostrar tableros		
Resumen	Este requerimiento permite mostrar el estado actual de ambos tableros después de cada turno, ya sea el tablero propio o el del rival.		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
Resultado o Postcondición	Muestra los tableros en la consola.		
Salidas	Nombre salida	Tipo de dato	Formato
	tableroHumano	int[][]	Ej. "0 2 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0"
			Ej. "0 2 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 3 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0"
	tableroMaquina	int[][]	

Identificador y nombre	<i>RF5: Determinar ganador</i>		
Resumen	<i>Este requerimiento permite verificar después de cada turno si todos los barcos de un jugador han sido hundidos para determinar el ganador.</i>		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
Resultado o Postcondición	Marca el juego como terminado y declara el ganador de la partida.		
Salidas	Nombre salida	Tipo de dato	Formato
	Mensaje de éxito1	String	<i>"GANASTE :D todos los barcos enemigos están hundidos"</i> <i>"Perdiste :(todos tus barcos fueron hundidos"</i>
	Mensaje de éxito2	String	"PUNTUACION ACTUAL: jugador: 0 maquina: 1"

Identificador y nombre	<i>RF6: Partidas múltiples</i>		
Resumen	<i>El sistema debe permitir al usuario jugar múltiples partidas en una misma ejecución del programa, manteniendo un registro de victorias y derrotas.</i>		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	menu	int	<i>Ej. "# Bienvenido a la simulación de Batalla Naval</i> <i># Estas son las opciones, ingresa:</i> <i># 1. - Para una partida personalizada</i> <i># 2. - para una partida Estándar</i> <i># 3. - Para ver el registro de partidas ganadas</i> <i># 4. - Para salir del programa</i> <i>> 2"</i>
Resultado o Postcondición	Se inicia una nueva partida, se muestran las estadísticas o se sale del programa según la selección del usuario.		
Salidas	Nombre salida	Tipo de dato	Formato
	partidaPersonalizada	String	<i>Ej. "# Ingresa tu nombre:</i> <i>Isa</i> <i># ¿Cuántos barcos deseas colocar? (Máx 10)</i> <i>1</i>

			# Barco 1: # Longitud del barco (1-5): 4 # Orientación: (H para horizontal, V para vertical): h # Coloca tu barco GUERRA (4 casillas, Horizontal) # Ingresa la coordenada X de GUERRA (1-10): 6 # Ingresa la coordenada Y de GUERRA (1-10): 8 # Finalmente la disposición de tu tablero con tus barcos queda así: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 # Información de colocación de barcos: # El GUERRA se colocará horizontalmente en las
--	--	--	---

			coordenadas: (6, 8), (7, 8), (8, 8), (9, 8). # === TU TURNO === # TUS BARCOS: # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 1 1 1 1 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # TUS ATAQUES AL ENEMIGO: # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # Dime la coordenada X a atacar en el tablero rival (1-10):"
--	--	--	--

	partidaEstandar	String	Ej. “# Bienvenido a la simulación de Batalla Naval para los estudiantes de Economía de la Universidad ICESI. # Te presentamos las siguientes opciones, ingresa: # 1. Para jugar # 2. Para conocer cuántas veces ha ganado el jugador y cuántas veces la máquina # 3. Para salir del programa > 1 # A continuación, se te pedirá que coloques tus barcos en el tablero. # Los barcos no deben solaparse y deben ajustarse a las reglas de orientación. # Indica las coordenadas X, Y para cada barco. Recuerda que el tablero es de 10x10. # Coloca tu Lancha (1 casilla): # Ingresar la coordenada X de la Lancha (1-10): > 3 # Ingresar la coordenada Y de la Lancha (1-10): > 5 # Coloca el Barco Médico (2 casillas, vertical): # Ingresar la coordenada X de la primera casilla del Barco Médico (1-10): > 7 # Ingresar la coordenada Y de la primera casilla del Barco Médico (1-10): > 2 # El Barco Médico se colocará verticalmente en las coordenadas (7, 2) y (7, 3). # Coloca el Barco de Provisiones (3 casillas,
--	-----------------	--------	--

			horizontal): # Ingresa la coordenada X de la primera casilla del Barco de Provisiones (1-10): > 4 # Ingresa la coordenada Y de la primera casilla del Barco de Provisiones (1-10): > 8 # El Barco de Provisiones se colocará horizontalmente en las coordenadas (4, 8), (5, 8) y (6, 8). # Coloca el Barco de Munición (3 casillas, vertical): # Ingresa la coordenada X de la primera casilla del Barco de Munición (1-10): > 9 # Ingresa la coordenada Y de la primera casilla del Barco de Munición (1-10): > 4 # El Barco de Munición se colocará verticalmente en las coordenadas (9, 4), (9, 5) y (9, 6). # Coloca el Buque de Guerra (4 casillas, horizontal): # Ingresa la coordenada X de la primera casilla del Buque de Guerra (1-10): > 1 # Ingresa la coordenada Y de la primera casilla del Buque de Guerra (1-10): > 9 # El Buque de Guerra se colocará horizontalmente en las coordenadas (1, 9), (2, 9), (3, 9) y (4, 9). # Coloca el Portaaviones (5 casillas, vertical): # Ingresa la coordenada
--	--	--	---

			X de la primera casilla del Portaaviones (1-10): > 6 # Ingresar la coordenada Y de la primera casilla del Portaaviones (1-10): > 1 # El Portaaviones se colocará verticalmente en las coordenadas (6, 1), (6, 2), (6, 3), (6, 4) y (6, 5). # Finalmente la disposición de tu tablero con tus barcos queda así: <pre># 0 0 0 0 0 1 0 0 0 0 # 0 0 0 0 0 1 1 0 0 0 # 0 0 0 0 0 1 1 0 0 0 # 0 0 0 0 0 1 0 0 1 0 # 0 0 1 0 0 1 0 0 1 0 # 0 0 0 0 0 0 0 0 1 0 # 0 0 0 0 0 0 0 0 0 0 # 0 0 0 1 1 1 0 0 0 0 # 1 1 1 1 0 0 0 0 0 0 # 0 0 0 0 0 0 0 0 0 0 # ¡Muy bien! ¡Ahora vamos a jugar! "</pre>
	estadísticas	String	Ej. "# === Estadísticas === # Partidas estándar: # Victorias humanas: 10 # Victorias máquina: 4 # Partidas personalizadas: # Victorias humanas: 4 # Victorias máquina: 2"
	salir	String	Ej. "# Muchas gracias por jugar. Hasta pronto."

Identificador y nombre	<i>RF7: Modos de juego (estándar/personalizada)</i>		
Resumen	<i>El sistema de ofrecer dos modos de juego: estándar (barcos predefinidos) y personalizada (el jugador elige la cantidad y tamaño de los barcos).</i>		
Entradas	Nombre entrada	Tipo de dato	Condición valores válidos
	menu	String	<i>Ej. “# Bienvenido a la simulación de Batalla Naval</i> <i># Estas son las opciones, ingresa:</i> <i># 1. - Para una partida personalizada</i> <i># 2. - para una partida Estándar</i> <i># 3. - Para ver el registro de partidas ganadas</i> <i># 4. - Para salir del programa”</i>
Resultado o Postcondición	Se configuran las reglas según el modo de juego seleccionado		
Salidas	Nombre salida	Tipo de dato	Formato
	Mensaje de éxito	String	<i>“Modo de juego seleccionado: Estandar”</i> <i>o</i> <i>“Modo de juego seleccionado: Personalizado”</i>
	Mensaje de error	String	<i>“Opcion inválida, intente de nuevo porfavor”</i>